Operation Kaskazi

CTF Forensics / Reverse Engineering Write-up

Challenge	Operation Kaskazi	Author	k4p3re
Difficulty	Medium	Points	300 (150 + 100 + 150)
Role	Incident-response analyst	Environment	Kali Linux (enos@kali), static analysis only
Artifacts	invoice_Q3_2026.eml, msupdate, capture.pcap	Flags	3 of 3 recovered

Scenario. A finance clerk at Zawadi Pay opened an overdue-account notice and nothing seemed to happen. Eleven days later the SOC caught the workstation beaconing to an unknown host, and data had already left the building. Three artifacts survived the reimage. The task is to reconstruct the intrusion and recover what was stolen.

Questions. Q1 (150 pts): How did they get in? Q2 (100 pts): What did the dropped tool hide? Q3 (150 pts): What did they steal?

1. Approach in one page

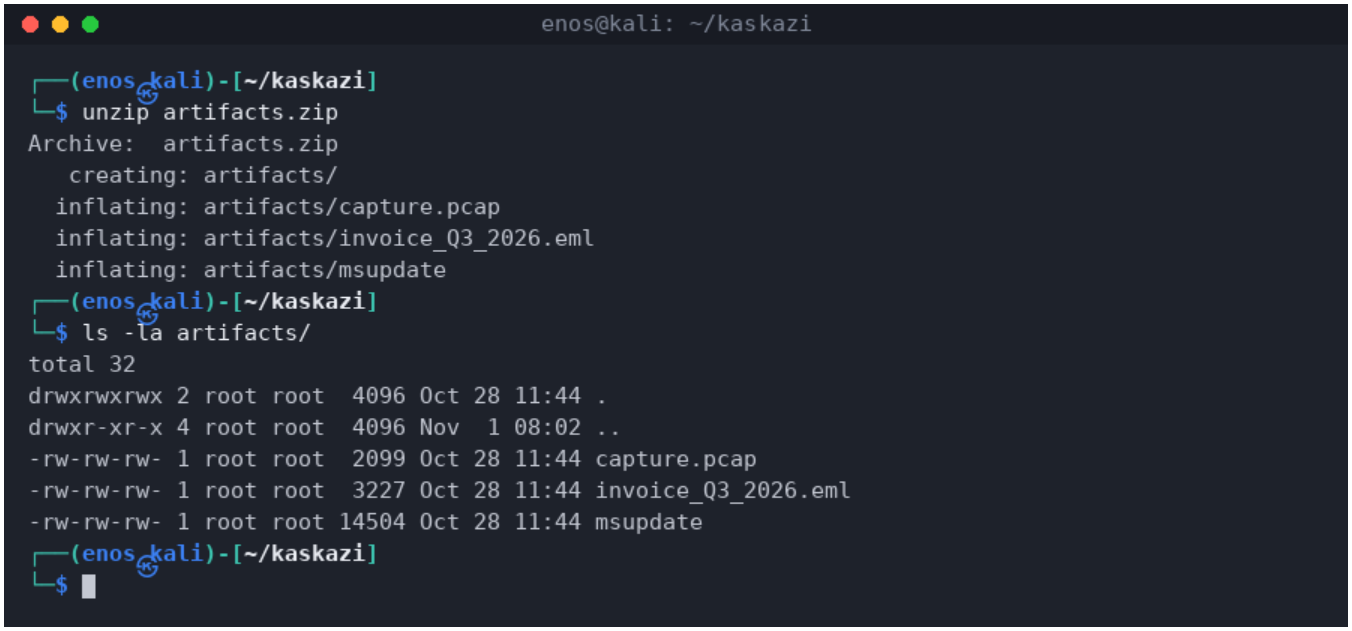
Each artifact maps to one stage of the intrusion, so the three questions chain together. The output of one stage supplies the key for the next:

Stage	Artifact	What it holds	What it gives the next stage
Q1 Initial access	invoice_Q3_2026.eml	Phishing mail with a JavaScript attachment (the lure)	Stage-1 config: drop path, C2 address, campaign tag
Q2 Dropped tool	msupdate (ELF)	RC4-encrypted config blob; key hidden as two XORed arrays	C2 session key for the network traffic
Q3 Exfiltration	capture.pcap	One TCP session, every message RC4-encrypted	The stolen file contents and the final flag

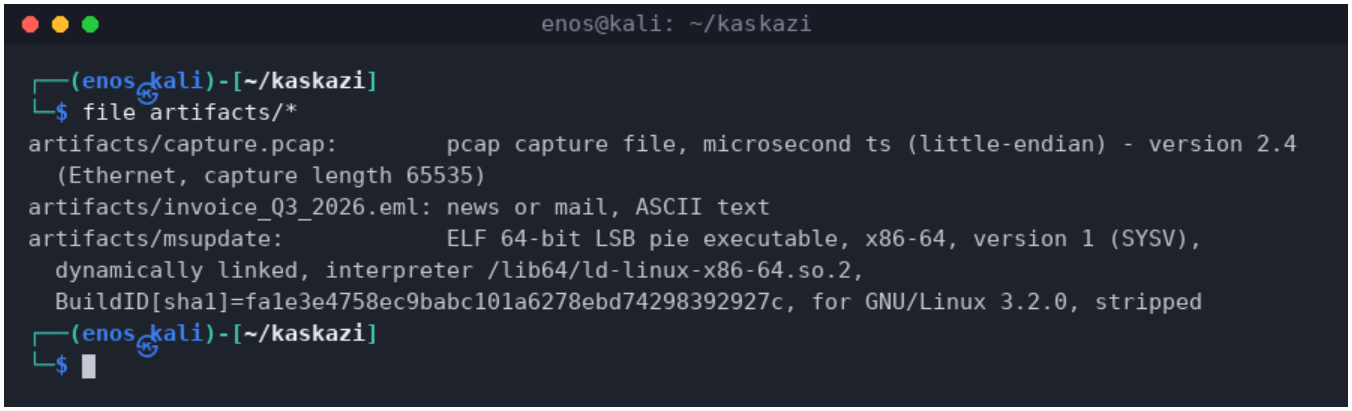
Safety note. Nothing in this challenge was executed. The JavaScript was decoded by reading it, the ELF was analysed with `strings`, `objdump` and Python, and the pcap was parsed byte by byte. That is the right default when handling any suspected malware: read it, do not run it.

2. Setup and triage

I created a working folder, unpacked the archive and identified every file before opening anything. `file` trusts the content, not the extension, so it is the first check on any unknown file.



Snapshot 1 - unpack the archive and list the three surviving artifacts.



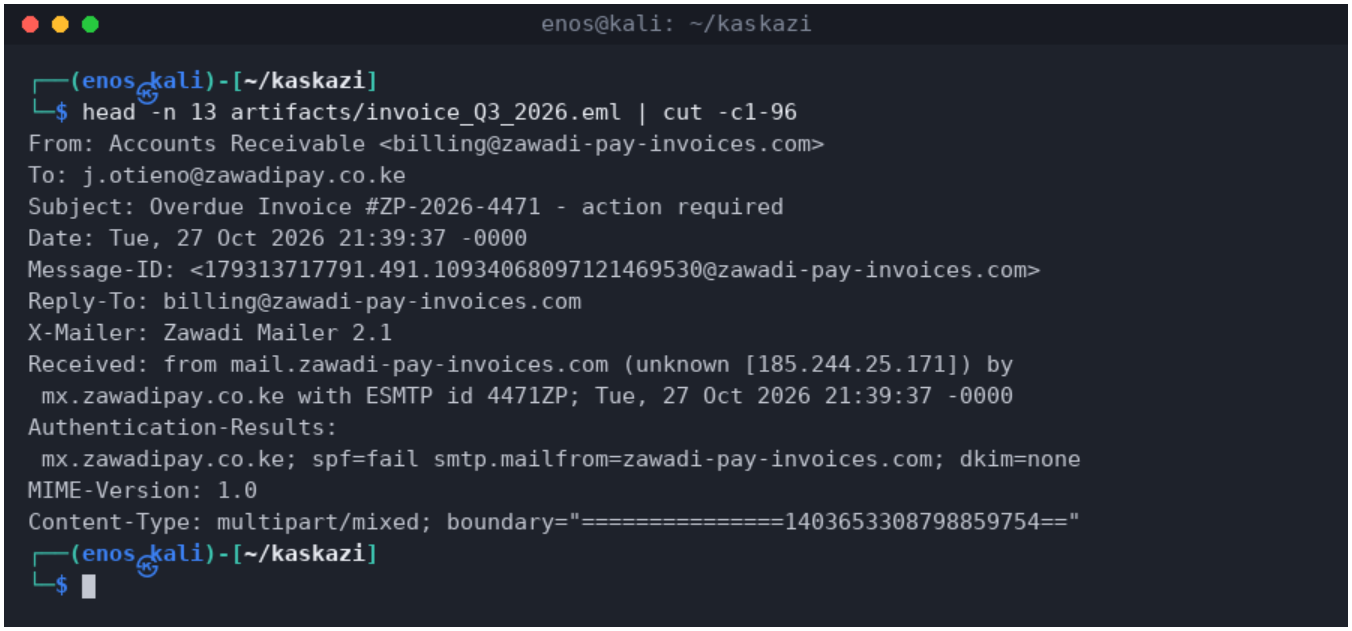
Snapshot 2 - identify each file by content: a pcap, a mail message and a 64-bit Linux ELF (stripped).

- **capture.pcap**: Ethernet capture, so I can parse it with plain Python or open it in Wireshark.
- **invoice_Q3_2026.eml**: a plain-text mail message, which will carry the phishing attachment.
- **msupdate**: a stripped ELF. No symbols, so function names will not help; I will read strings and disassembly.

3. Q1 - How did they get in? (150 pts)

3.1 Read the mail headers

The headers say who the mail claims to be from and what the receiving server actually saw.



Snapshot 3 - top of the .eml file. The sender domain is zawadi-pay-invoices.com, not the victim's own domain.

The lower header lines (Received and Authentication-Results) give the real picture: the message came from `185.244.25.171`, and the receiving server recorded `spf=fail` and `dkim=none`. The body pushes urgency ("overdue", "settle today") and tells the clerk to "enable content", the classic social-engineering pattern.

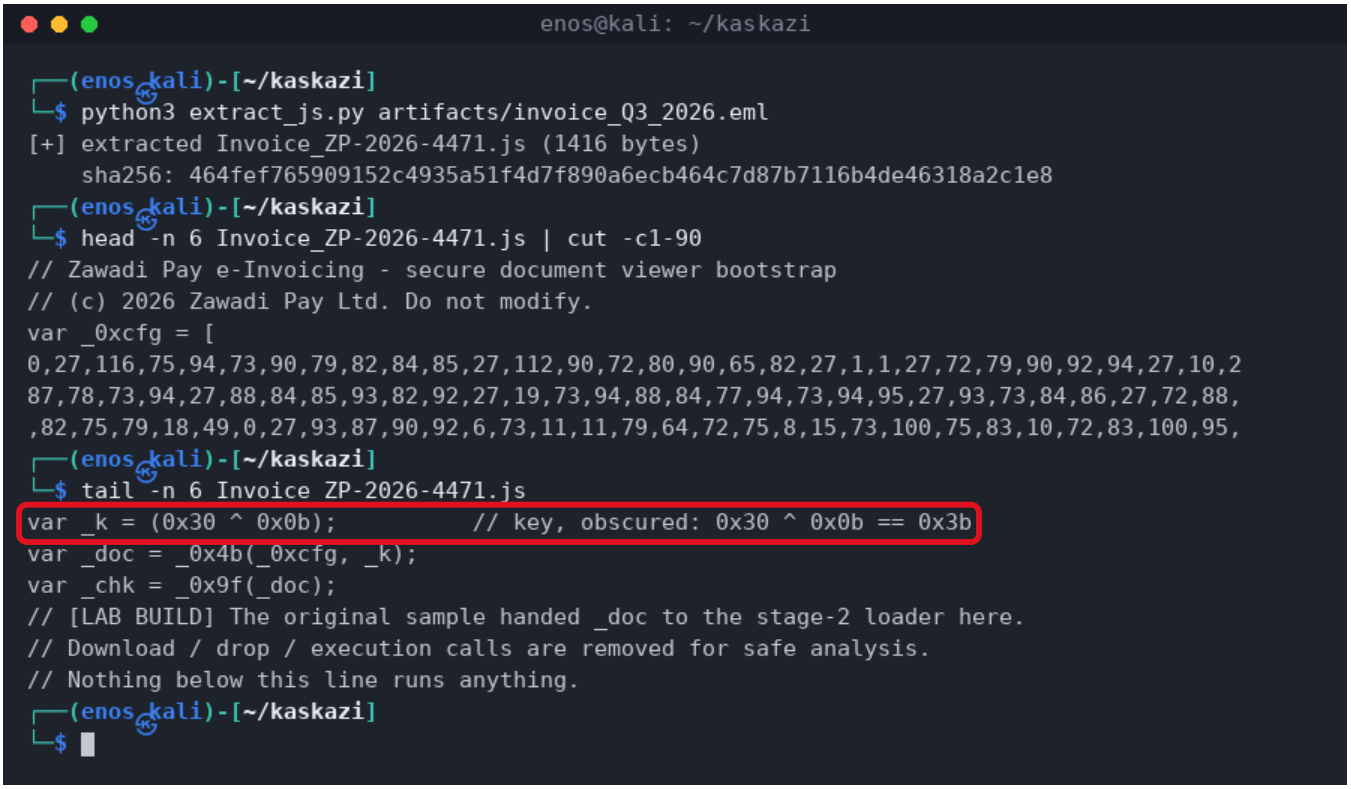
3.2 Extract the attachment (without opening it)

The attachment is base64 inside the MIME body. Python's `email` module decodes it for me and writes it to disk as an inert text file.

```
import email, hashlib, sys
from email import policy

m = email.message_from_file(open(sys.argv[1]), policy=policy.default)
for part in m.iter_attachments():
    name = part.get_filename()
    data = part.get_payload(decode=True)
    open(name, "wb").write(data)
    print(f"[+] extracted {name} ({len(data)} bytes)")
    print(f"    sha256: {hashlib.sha256(data).hexdigest()}")
```

extract_js.py - pull the MIME attachment out of the .eml and print its hash.



Snapshot 4 - the attachment is Invoice_ZP-2026-4471.js, a script posing as a secure viewer. Note the big integer array and the decoy checksum function.

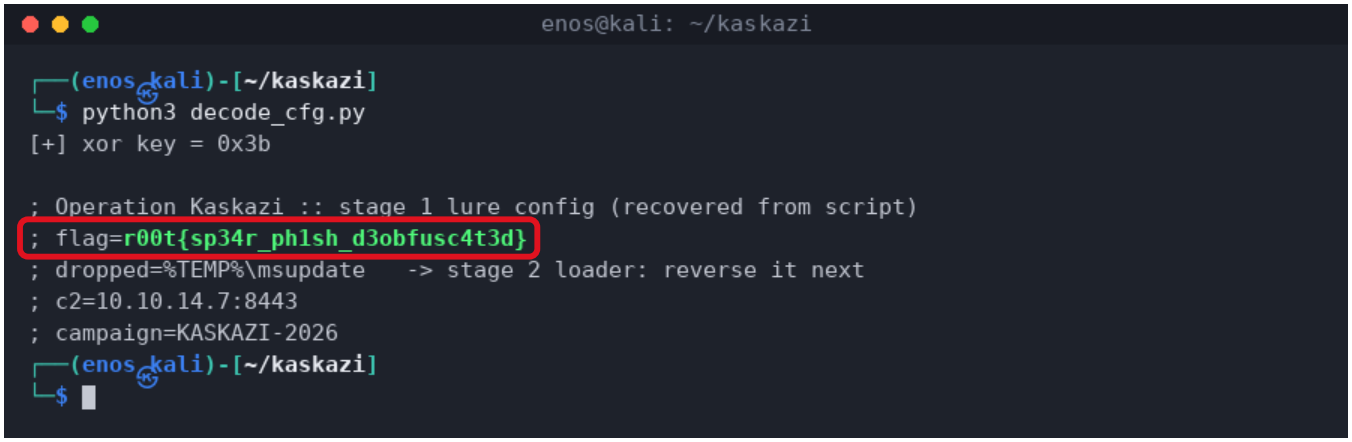
3.3 De-obfuscate the script by reading it

Three things stand out in the script: a large integer array `_0xcfg`, a small function that XORs each element with a key, and the line `var _k = (0x30 ^ 0x0b)`. The key is written as an expression so a quick glance does not reveal it, but it evaluates to `0x3b`. The checksum function `_0x9f` does nothing useful; it is a decoy. The lab build also stripped the download-and-execute calls, so decoding the array is all that is needed.

```
import re

js  = open("Invoice_ZP-2026-4471.js").read()
arr = [int(x) for x in re.search(r"_0xcfg = \[(.*?)\];", js, re.S)
       .group(1).replace("\n", "").split(",") if x.strip()]
key = 0x30 ^ 0x0b                    # the "obscured" key from the script
print(f"[+] xor key = {key:#04x}\n")
print(bytes(b ^ key for b in arr).decode())
```

decode_cfg.py - re-implement the script's XOR decoder in Python.



Snapshot 5 - the decoded stage-1 config: flag, drop path (%TEMP%\msupdate), C2 10.10.14.7:8443 and campaign tag KASKAZI-2026.

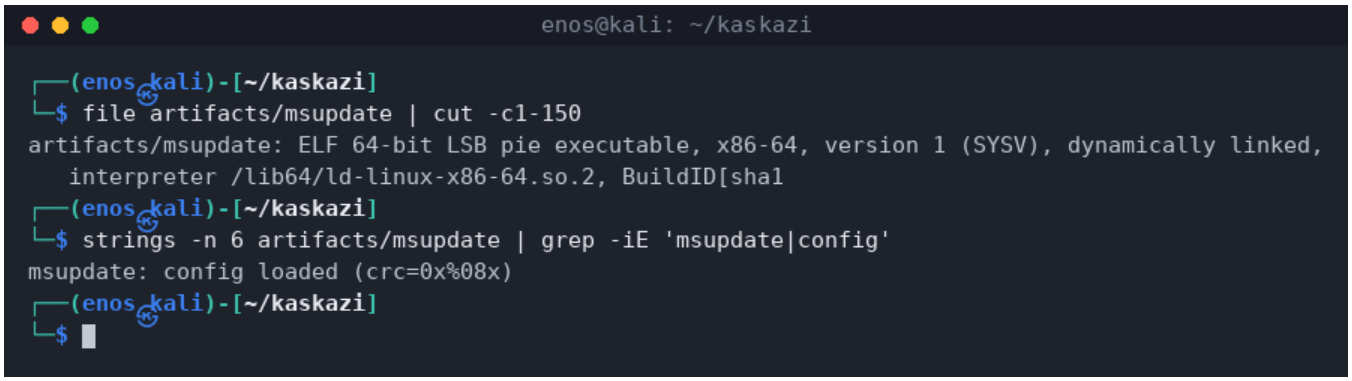
Q1 flag	`r00t{sp34r_ph1sh_d3obfusc4t3d}`

Answer to Q1: spearphishing with a malicious script attachment (MITRE T1566.001), disguised as an overdue invoice from a lookalike domain, with the payload hidden behind a single-byte XOR.

4. Q2 - What did the dropped tool hide? (100 pts)

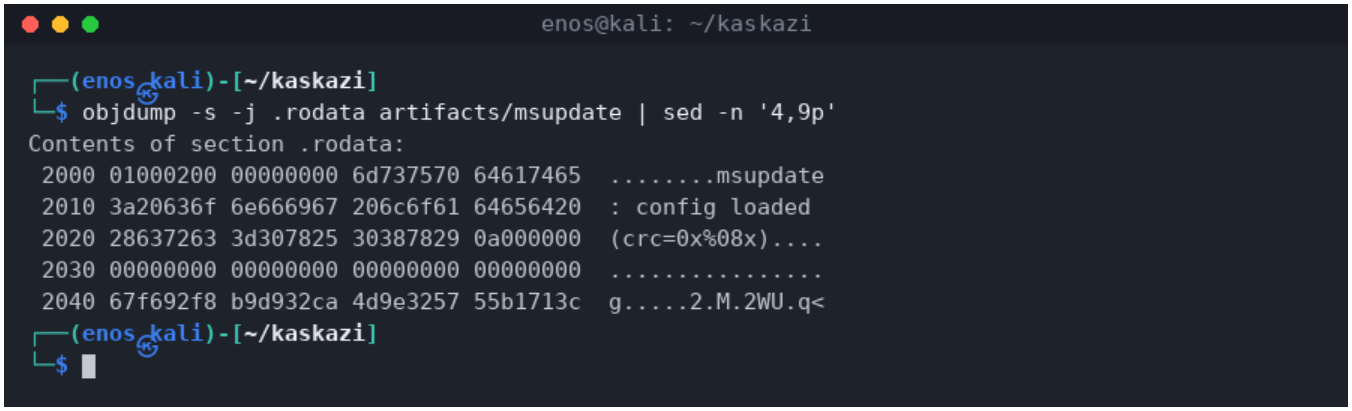
The Q1 config says the lure drops `msupdate` and tells me to reverse it next. That name imitates a Microsoft update binary, a common masquerading trick (T1036).

4.1 First look: strings and sections



Snapshot 6 - the only readable program strings are a status message. Anything useful must be encoded.

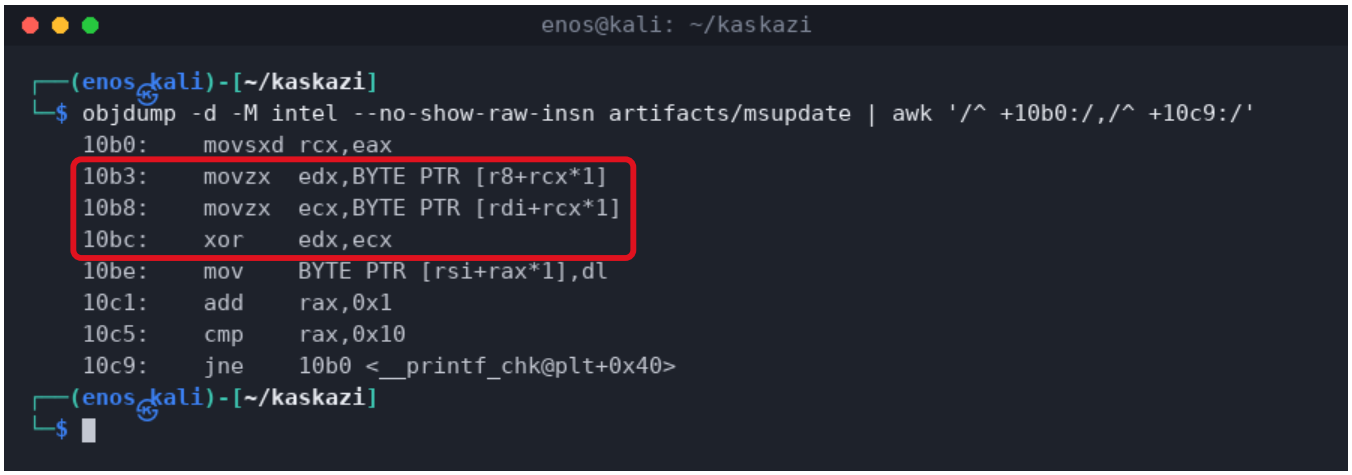
Almost no readable text means the interesting data is encoded. I dumped the read-only data section next.



Snapshot 7 - .rodata. Below the printf string, from 0x2040, is a block of random-looking bytes: an encrypted blob.

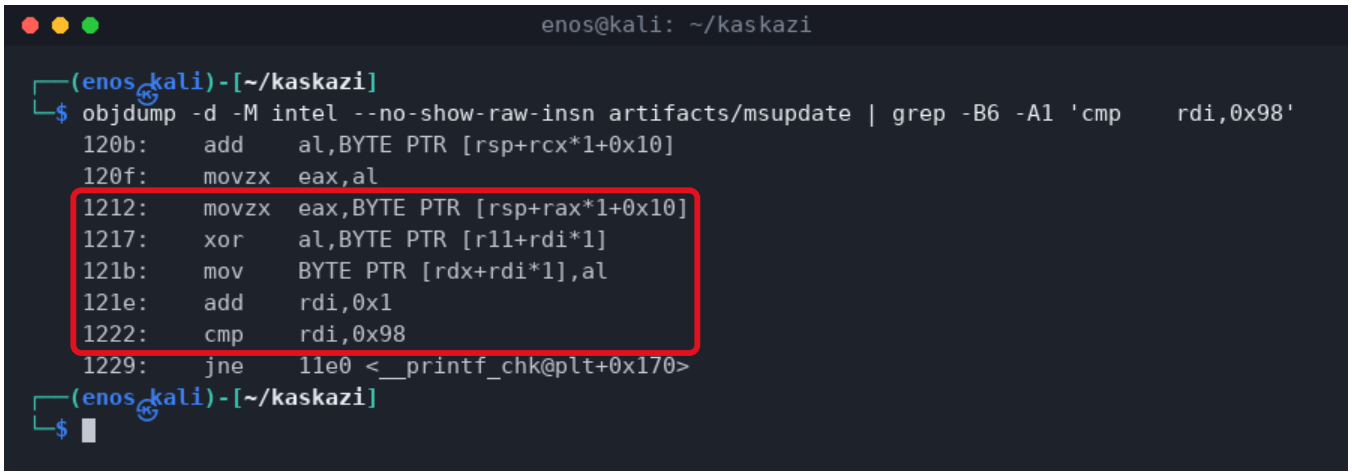
4.2 Disassemble to find how it is decrypted

Two loops matter. The first runs 16 times and XORs two byte arrays into a 16-byte buffer on the stack. That buffer is a key built at runtime, so the key never appears as a string.



Snapshot 8 - key derivation: load a byte from each of two arrays (r8 and rdi), XOR them (10bc), store the result, loop until 0x10 (16) bytes.

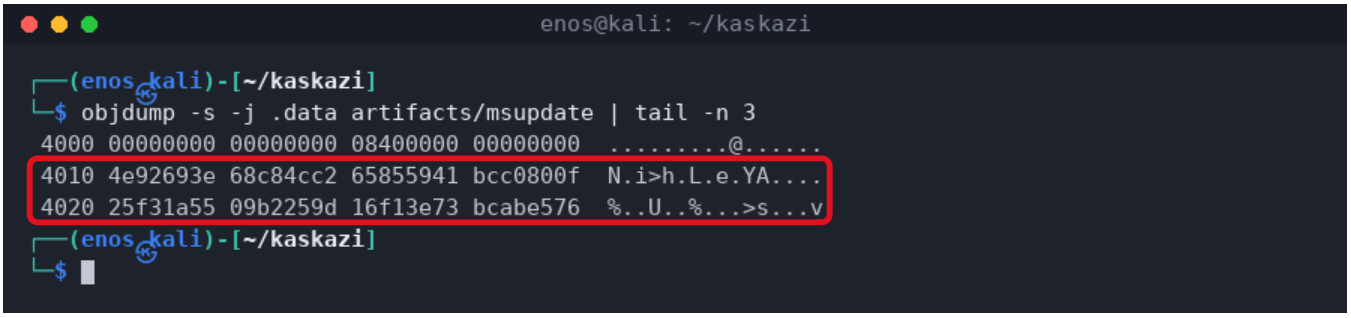
The second loop ends with `cmp rdi,0x98`. It XORs each blob byte with a stream byte, 0x98 (152) times, and the stream is produced by indexing a 256-entry array that is being swapped around. That is the shape of the RC4 PRGA. The earlier loop that fills the 256-entry array and mixes in the key is the RC4 KSA.



Snapshot 9 - the decrypt loop: S-box lookup (1212), XOR with the ciphertext byte (1217), store, increment, stop at 0x98 bytes.

4.3 Get the key material and decrypt

The two key arrays live in `.data` at virtual addresses 0x4010 and 0x4020.



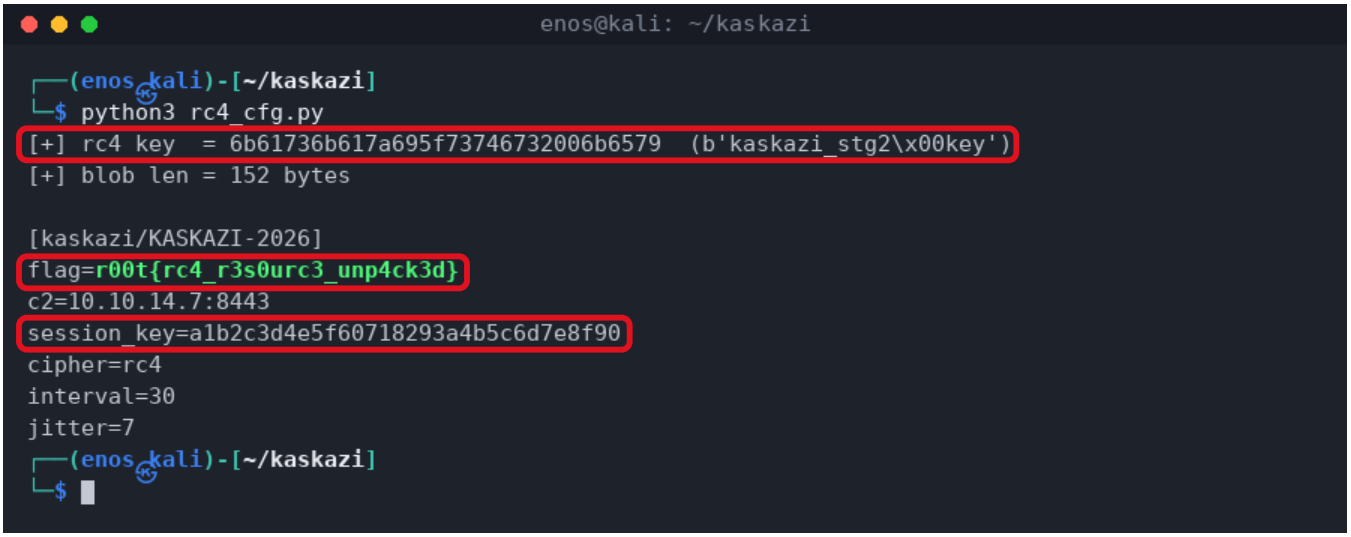
Snapshot 10 - .data. Two 16-byte arrays at 0x4010 and 0x4020; neither is the key on its own.

Section headers show `.data` mapped at virtual address 0x4000 from file offset 0x3000, so the arrays are at file offsets 0x3010 and 0x3020. .rodata maps 1:1 (0x2040 is both). XORing the arrays gives the key `kaskazi_stg2\x00key`.

```
def rc4(key, data):
    S, j = list(range(256)), 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) & 255
        S[i], S[j] = S[j], S[i]
    i = j = 0
    out = bytearray()
    for c in data:
        i = (i + 1) & 255
        j = (j + S[i]) & 255
        S[i], S[j] = S[j], S[i]
        out.append(c ^ S[(S[i] + S[j]) & 255])
    return bytes(out)

d   = open("artifacts/msupdate", "rb").read()
a   = d[0x3010:0x3020]                # .data  @ vaddr 0x4010
b   = d[0x3020:0x3030]                # .data  @ vaddr 0x4020
key = bytes(x ^ y for x, y in zip(a, b))
blob = d[0x2040:0x2040 + 0x98]        # .rodata @ 0x2040, 0x98 bytes
print(f"[+] rc4 key  = {key.hex()}  ({key!r})")
print(f"[+] blob len = {len(blob)} bytes\n")
print(rc4(key, blob).decode())
```

rc4_cfg.py - rebuild the key, run RC4 over the 0x98-byte blob, print the plaintext.



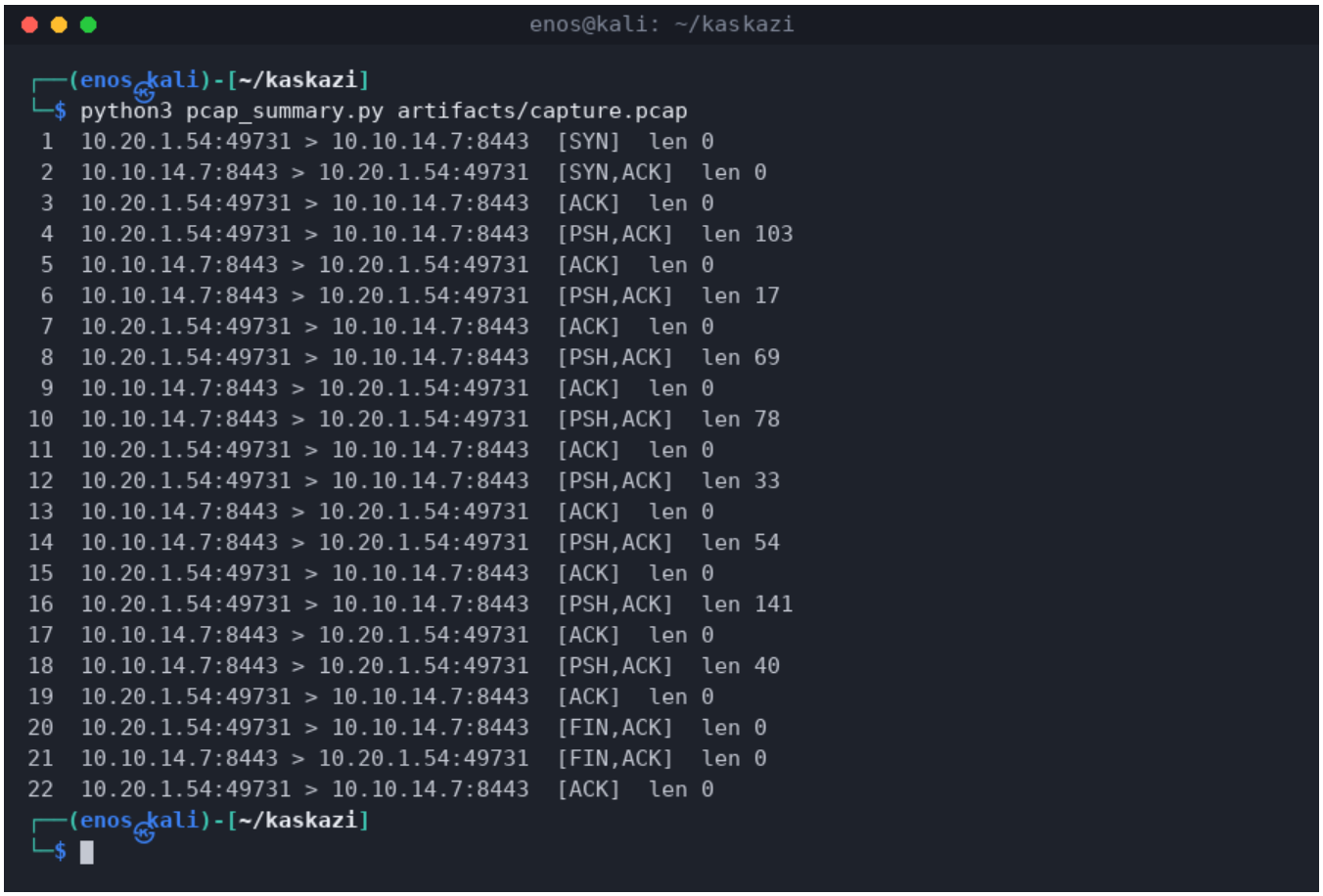
Snapshot 11 - decrypted config: flag, C2 address, a 16-byte session key, cipher=rc4 and beacon timing (30 s interval, 7 s jitter).

Q2 flag	**`r00t{rc4_r3s0urc3_unp4ck3d}`**

Answer to Q2: the implant hides an RC4-encrypted configuration blob in its read-only data. The key is split into two arrays so it never exists in the file as a string. The config reveals the C2 and, importantly, the **session key** needed for Q3.

5. Q3 - What did they steal? (150 pts)

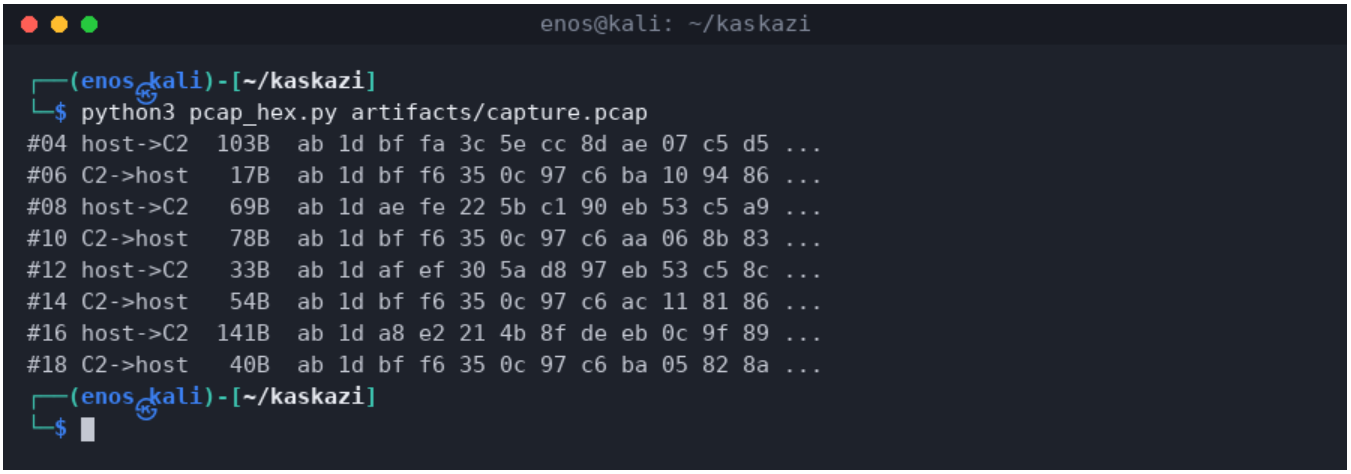
5.1 Map the conversation



Snapshot 12 - one TCP session from the workstation 10.20.1.54 to the C2 10.10.14.7:8443. Handshake, six data exchanges, then teardown.

There is only one flow, to the exact C2 address and port from the configs. The data packets are the PSH,ACK ones; their payloads are unreadable, so they are encrypted. Q2 already gave me `cipher=rc4` and a session key.

5.2 Work out the framing



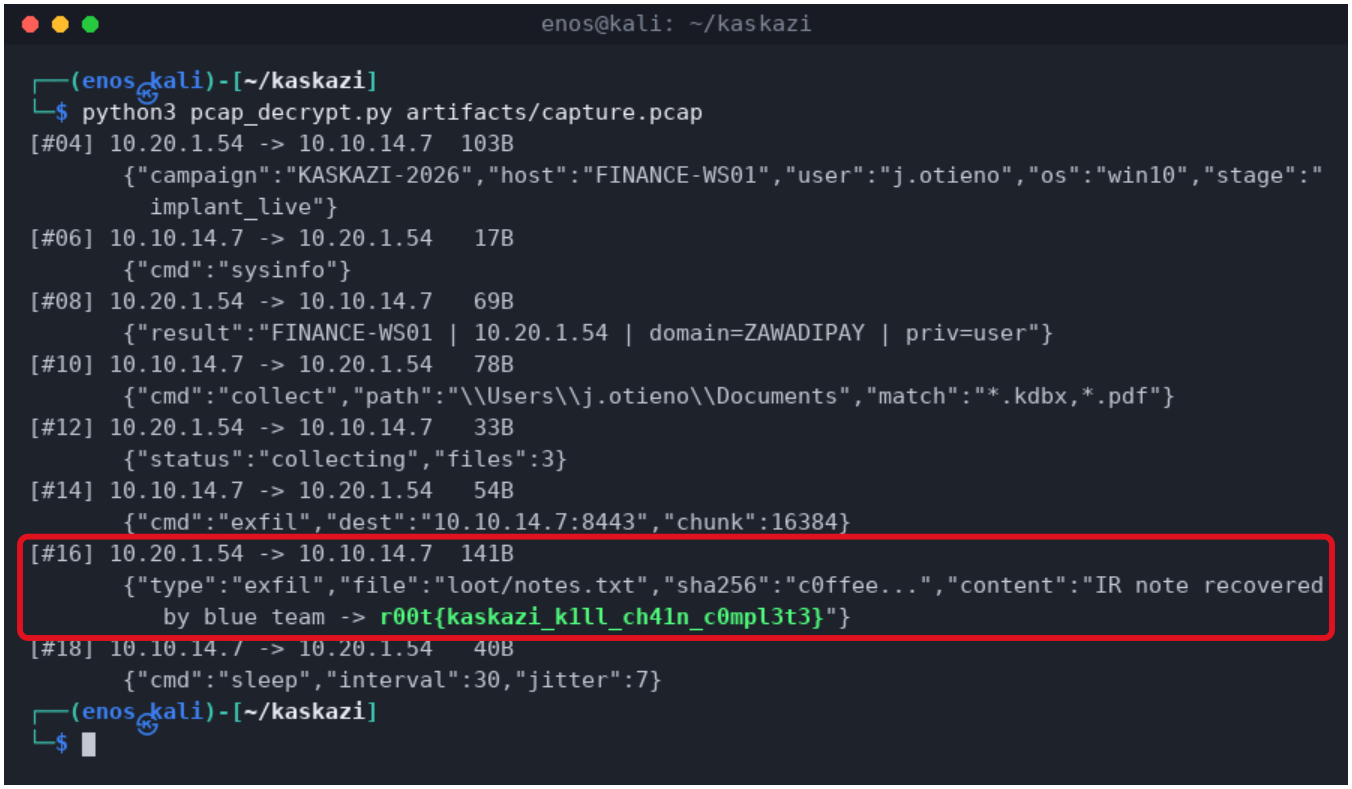
Snapshot 13 - first 12 bytes of each payload. Every C2-to-host message starts with the same 8 bytes (ab 1d bf f6 35 0c 97 c6).

That repeated prefix is the clue. Two messages with the same first bytes can only come from the same plaintext start encrypted with the same keystream start. So the cipher is restarted for each message (a fresh RC4 per packet), and the commands share a common beginning. A single running keystream would make every packet look different. So: decrypt each payload independently, with no header to skip.

```
import struct, sys
from rc4lib import rc4

SESSION_KEY = bytes.fromhex("a1b2c3d4e5f60718293a4b5c6d7e8f90")   # from msupdate config
data, off, n = open(sys.argv[1], "rb").read(), 24, 0
while off < len(data):
    _, _, caplen, _ = struct.unpack("<IIII", data[off:off + 16]); off += 16
    pkt = data[off:off + caplen]; off += caplen; n += 1
    ip  = pkt[14:]; ihl = (ip[0] & 15) * 4
    tcp = ip[ihl:]; payload = tcp[(tcp[12] >> 4) * 4:]
    if not payload:
        continue
    src = ".".join(map(str, ip[12:16])); dst = ".".join(map(str, ip[16:20]))
    print(f"[#{n:02d}] {src} -> {dst}  {len(payload):>3}B")
    print("      ", rc4(SESSION_KEY, payload).decode())     # fresh keystream per message
```

pcap_decrypt.py - for each data packet, run RC4 with the session key from Q2 and print the JSON.



Snapshot 14 - the full C2 conversation, decrypted. The final host-to-C2 message exfiltrates loot/notes.txt and carries the flag.

Q3 flag	`r00t{kaskazi_k1ll_ch41n_c0mpl3t3}`

Msg	Direction	Meaning
#04	host to C2	Check-in: campaign KASKAZI-2026, host FINANCE-WS01, user j.otieno, stage implant_live
#06 / #08	C2 to host / reply	sysinfo command; reply shows the host is a domain-joined machine with user privileges only
#10 / #12	C2 to host / reply	collect *.kdbx,*.pdf from the user's Documents folder; host reports 3 files
#14 / #16	C2 to host / reply	exfil in 16 KB chunks to the C2; host uploads loot/notes.txt (carries the flag)
#18	C2 to host	sleep: beacon every 30 s with 7 s jitter (matches the config from Q2)

Answer to Q3: the attacker collected KeePass databases (.kdbx) and PDFs from the clerk's Documents folder and exfiltrated them over the encrypted C2 channel. The challenge file inside the capture, `loot/notes.txt`, holds the flag.

6. Reconstructed intrusion

Phase	What happened	Evidence	MITRE
Initial access	Lookalike-domain invoice mail with a .js attachment to j.otieno	.eml headers: SPF fail, DKIM none, sender IP 185.244.25.171	T1566.001
Execution	Clerk opens the "secure viewer" script	Obfuscated JS, XOR key 0x3b, stage-1 config	T1204.002, T1059.007

Phase	What happened	Evidence	MITRE
Defence evasion	Single-byte XOR in the lure; decoy checksum; custom RC4 with a split key in the loader	Snapshots 5, 8, 9	T1027, T1140
Masquerading	Dropper placed as msupdate in %TEMP%	Stage-1 config	T1036.005
Command and control	Beaconing to 10.10.14.7:8443, RC4-encrypted over a non-standard port	pcap, 30 s interval, 7 s jitter	T1573.001, T1571, T1029
Discovery	sysinfo	Message #06 / #08	T1082
Collection	Search Documents for *.kdbx and *.pdf	Message #10 / #12	T1005
Exfiltration	16 KB chunks over the C2 channel	Message #14 / #16	T1041

Indicators of compromise

Type	Value
Sender domain	`zawadi-pay-invoices.com`
Sender IP	`185.244.25.171`
Attachment	`Invoice_ZP-2026-4471.js (sha256 464fef76...a2c1e8)`
Dropped file	`%TEMP%\msupdate`
C2	`10.10.14.7:8443 (RC4; session key a1b2c3d4e5f60718293a4b5c6d7e8f90)`
Victim	`FINANCE-WS01 / 10.20.1.54 / user j.otieno`
Campaign tag	`KASKAZI-2026`

7. Key takeaways and lessons

For the analyst (techniques that transfer to other challenges)

- **Identify before you inspect.** `file` and `strings` take seconds and decide the whole approach. Here, almost no strings in the ELF meant the data was encoded.
- **Read, do not run.** Static analysis answered every question. Rebuilding a decoder in Python is safer than detonating a sample and gives you the same result.
- **Recognise algorithm shapes.** A 256-entry array, swaps, and a modular index are RC4 almost every time. A loop that runs exactly 0x10 times and XORs two arrays is a key being assembled.
- **Mind addresses versus file offsets.** Virtual address 0x4010 was file offset 0x3010 in this binary. Check section headers before slicing bytes with a script.
- **Repeated ciphertext prefixes leak structure.** Identical first bytes in different packets showed that the keystream restarted per message and that commands share a common start.
- **Chain your findings.** Each stage gave me the next key: lure config to the dropped binary to the session key to the traffic. When stuck, ask what an earlier artifact told me that I have not used yet.
- **Ignore decoys.** The checksum function in the JS and the benign strings in the ELF were there to waste time. Follow the data flow, not the function names.

For the defender (what would have stopped or detected this)

- **Block script attachments** (.js, .vbs, .wsf) at the mail gateway, and set Windows Script Host to open scripts in a text editor or disable it for users who do not need it.
- **Enforce SPF, DKIM and DMARC.** The mail failed SPF and had no DKIM, yet it reached a finance inbox. Quarantine or visibly tag such mail, and flag lookalike domains.
- **Train for urgency.** "Overdue" and "enable content" together are a red flag. Give staff an easy "report phishing" button.
- **Watch for odd egress.** A workstation talking to an unknown host on port 8443 with a steady 30 s beat is a classic beacon. Egress filtering and proxy logging, plus beacon-detection rules on interval and jitter, would have caught the implant days earlier.
- **Protect password vaults.** KeePass databases in Documents were the prize. Use a managed password solution with MFA and keep vault files out of user shares.
- **Hunt on masquerading.** A process named like an update tool running from %TEMP% should alert.
- **Do not rely on obfuscation or home-grown crypto** (XOR, RC4 with a split key). It only slows down analysis; every layer here was reversible from the files alone.

8. Flags

Question	Points	Flag
Q1 How did they get in?	150	`r00t{sp34r_ph1sh_d3obfusc4t3d}`
Q2 What did the dropped tool hide?	100	`r00t{rc4_r3s0urc3_unp4ck3d}`
Q3 What did they steal?	150	`r00t{kaskazi_k1ll_ch41n_c0mpl3t3}`

Reproduce it: unzip the artifacts, then run `python3 extract_js.py`, `decode_cfg.py`, `rc4_cfg.py` and `pcap_decrypt.py` in that order. Each script prints its flag.